

Computational & Problem Solving Using C

Complete 4-Unit Lecture Notes & University Solved Examination Bank

Course Code: BCSE-008 / C Programming

Department: B.Tech Computer Science & Engineering

Units Covered: Units 1, 2, 3, and 4 (Exhaustive Theory + Solved Problems)

Visuals Included: ANSI Flowcharts, Vector Circuit Diagrams & Schematics

Mathematical Rigor: Publication-grade KaTeX typesetting

Programming Fundamentals, Compilation Pipeline & Problem Solving

Exam Weightage: 24 - 28 Marks • **Estimated Study Time:** 40 mins

Computational problem classifications, Deductive vs Inductive reasoning, Top-down design & flowchart standards, Black-box vs White-box testing, 4 error tiers, Big-O complexity matrix, dry-run trace tables, translators comparison, 4-stage compilation pipeline, and virtual process memory layout.

1. Computational Problem Classification & Logic Types

1. Computational Problem Classifications

In computer science and software engineering, computational problems are systematically categorized based on the nature of their output space:

- 1. Decision Problems:** Problems requiring a deterministic Boolean output (`True` or `False` , `1` or `0`). Example: Given an integer N , decide whether N is prime.
- 2. Counting Problems:** Problems that compute the cardinality of valid solutions satisfying specific constraints. Example: Counting the number of distinct paths in a grid.
- 3. Optimization Problems:** Problems requiring the identification of the globally best candidate among all feasible configurations according to an objective function. Example: Shortest path routing in network packet switching.

2. Logic Paradigms: Deductive vs. Inductive Reasoning

Program construction and formal algorithmic proofs rely on two core logic frameworks:

#	Dimension	Deductive Logic	Inductive Logic
1	Direction	Top-Down: Starts with general axioms/theories and deduces specific guaranteed truths.	Bottom-Up: Starts with specific empirical observations and infers general probabilistic patterns.
2	Validation Guarantee	Guaranteed Absolute Truth: If premises are valid and logic rules hold, conclusions are incontrovertible.	Probabilistic / Plausible: True for observed training sample, but unverified for unobserved edge cases.
3	Software Role	Formal program verification, loop invariants, and algorithmic correctness proofs.	Test-driven development (TDD), heuristic AI search, and empirical profiling.

| **4 | C Engineering Example** | Proving loop invariant: If loop starts at `i=0` and increments by 1, `while(i < N)` executes exactly N iterations. | Running 10 unit test cases with positive inputs: it suggests the code works, but does not mathematically prove absence of memory leaks. |

2. Algorithmic Design, Flowcharts & Stepwise Refinement

1. Stepwise Refinement (Top-Down Decomposition)

Stepwise Refinement (formulated by Niklaus Wirth) is an architectural discipline where a high-level problem statement is iteratively decomposed into smaller, self-contained sub-modules until every sub-task maps directly to elementary language primitives.

✓ **FUNDAMENTAL LAW / RULE:**

Core Benefits of Stepwise Refinement:

- **Cognitive Manageability:** Prevents cognitive overload by isolating architectural concerns.

- **Modularity:** Subroutines can be developed, compiled, and unit-tested in isolation.

- **Reusability:** Common computational kernels (e.g., searching, sorting) become shared utility functions.

2. Standard ANSI Flowcharting Symbols

ANSI standardized flowchart symbology enforces unambiguous visual algorithmic blueprints:

Symbol Shape	ANSI Geometric Form	Engineering Purpose	C Language Mapping
Terminal	Rounded Oval / Capsule	Indicates entry point (Start) or termination point (Stop/End).	<code>int main() / return 0;</code>
Input / Output	Parallelogram	Data ingestion or presentation across peripheral streams.	<code>scanf() , printf() , fgets()</code>
Process	Rectangle	Computational operations, variable initializations, state mutations.	<code>c = a + b; , i = 0;</code>
Decision	Rhombus (Diamond)	Conditional evaluation producing branching Boolean pathways (True/False).	<code>if (x > 0) , switch (ch)</code>

| **Connector** | Circle with alphanumeric key | Merges convergent execution paths without crossing connector arrows. | Loop junction nodes |

3. Testing Strategies & 4 Tiers of Program Defects

1. Black-Box vs. White-Box Testing Methodologies

Quality assurance in compiled systems incorporates two complementary testing paradigms:

• **Black-Box Testing (Functional Testing):**

- Assesses software behavior purely against specification inputs and expected outputs.
- The tester has **zero visibility** into the internal source code architecture.
- Primary Techniques: **Equivalence Class Partitioning** and **Boundary Value Analysis (BVA)**.

• **White-Box Testing (Structural Testing):**

- Inspects internal control structures, branches, and execution paths with full source code visibility.
- Primary Metrics: **Statement Coverage**, **Branch/Decision Coverage**, and **Cyclomatic Complexity**.

2. The 4 Tiers of Programming Defects

Software defects are classified into four distinct phases of the software lifecycle:

Error Tier	Detection Phase	Root Cause & Characteristics	C Concrete Example
1. Syntax Error	Translation / Compile Time	Violation of C grammar rules; caught by compiler tokenizer or parser.	Missing semicolon <code>;</code> , mismatched braces <code>{</code> , misspelled keyword <code>whlie</code> .
2. Linker Error	Linking Time (ld)	Unresolved external symbols; function declared but missing object definition.	Calling <code>clrscr()</code> or <code>foo()</code> without linking their object library.
3. Runtime Error	Program Execution Time	Illegal CPU instructions or OS protection faults causing abnormal termination.	Division by zero (<code>x / 0</code>), Segmentation Fault (SIGSEGV) accessing address 0x0.

| 4. Semantic / Logic Error | User Testing / Output Validation | Program compiles and runs without crashing, but generates incorrect results. | Using `i <= n` instead of `i < n` causing an off-by-one array overwrite. |

4. Algorithmic Complexity (Big-O) & Dry-Run Trace Tables

1. Asymptotic Complexity: Big-O Mathematical Taxonomy

Big-O notation characterizes the upper bound of resource consumption (time or memory space) as input size *N* tends to infinity:

Notation	Complexity Name	Growth Rate for $N = 1000$	Canonical C Algorithm
O(1)	Constant Time	1 operation	Array indexing <code>A[i]</code> , pointer dereference <code>*p</code> .
O(log N)	Logarithmic Time	≈ 10 operations	Binary search in sorted array (halves problem space each step).
O(N)	Linear Time	1,000 operations	Linear sequential search, single-pass array traversal.
O(N log N)	Linearithmic Time	$\approx 10,000$ operations	Merge Sort, Quick Sort (average case divide & conquer).
O(N²)	Quadratic Time	1,000,000 operations	Bubble Sort, Selection Sort, nested 2D matrix iteration.

2. Dry-Run Variable Execution Trace Table

A **Dry Run** is a manual code simulation tracing variable states against line numbers to expose logic defects:

```
int sum = 0;
for (int i = 1; i <= 3; i++) {
    sum += i;
}
```

Trace Table:

Step	Source Line	Variable <code>i</code>	Condition <code>i <= 3</code>	Variable <code>sum</code>	Step Action Summary
0	Line 1	Undefined	-	0	Initialization of accumulator <code>sum = 0</code> .
1	Line 2	1	<code>1 <= 3</code> (True)	0	Loop init <code>i = 1</code> ; entry condition holds.
2	Line 3	1	-	1	Accumulate: <code>sum = 0 + 1 = 1</code> .
3	Line 2	2	<code>2 <= 3</code> (True)	1	Increment <code>i++</code> ; condition holds.
4	Line 3 {2}	-	3	Accumulate: <code>sum = 1 + 2 = 3</code> .	
5	Line 2	3	<code>3 <= 3</code> (True)	3	Increment <code>i++</code> ; condition holds.
6	Line 3	3	-	6	Accumulate: <code>sum = 3 + 3 = 6</code> .

5. Translators, 4-Stage Compilation Pipeline & Process Memory Layout

1. Translators: Compiler vs. Interpreter vs. Assembler

Comparative Dimension	Compiler	Interpreter	Assembler
Input Source	High-level language (C, C++, Rust).	High-level script (Python, JS, Ruby).	Assembly language mnemonics (8086, x86).
Translation Unit	Scans entire translation unit at once.	Interprets statement-by-statement.	1-to-1 conversion of mnemonic to opcode.
Intermediate Object	Generates relocatable object file (<code>.o</code> / <code>.obj</code>).	No standalone object file produced.	Generates relocatable machine code.
Execution Speed	Extremely fast direct hardware execution.	Slower due to runtime interpretation overhead.	Maximum execution speed on target CPU.
Error Reporting	Compiles list of all syntax errors at once.	Halts execution immediately on first error.	Reports syntax errors during assembly pass.

2. The 4-Stage C Compilation Pipeline

When invoking `gcc prog.c -o prog.exe` , four distinct system subsystems execute in pipeline order:

```
[ prog.c ] (Source Text)
|
▼ (1. Preprocessor: cpp)
[ prog.i ] (Expanded Pure C: stripped comments, expanded #include, macro constants)
|
▼ (2. Compiler: ccl)
[ prog.s ] (Assembly Code: architecture-specific target CPU instructions)
|
▼ (3. Assembler: as)
[ prog.o / .obj ] (Relocatable Machine Binary: binary CPU opcodes with unresolved addresses)
|
▼ (4. Linker: ld + libc.a / CRT)
[ prog.exe / a.out ] (Executable Binary: fully resolved memory addresses, ready to load into RAM)
```

3. C Process Virtual Memory Layout in RAM

When an executable binary is launched by the OS loader, it is allocated a private virtual memory address space:

Memory Segment	Growth Direction	Stored Content & Characteristics
Stack Segment	Grows Downwards (High → Low)	Local automatic variables (<code>auto</code>), function parameter lists, return addresses, and stack activation records. Automatically allocated and released.
Free Pool Boundary	Stack/Heap Interface	Memory area where stack and heap grow toward each other. Depletion causes Stack Overflow.
Heap Segment	Grows Upwards (Low → High)	Dynamically allocated memory at runtime via <code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> . Must be explicitly released via <code>free()</code> .
BSS Segment	Static / Fixed	Uninitialized global and static variables. Automatically initialized to zero by the OS loader prior to <code>main()</code> .
Initialized Data	Static / Fixed	Global and static variables explicitly initialized with non-zero literals in source code (e.g., <code>int global_x = 100;</code>).

| **Text / Code Segment** | Static / Low Memory (0x0) | Read-only compiled CPU binary instructions. Protected by OS from runtime overwrite attempts. |

6. University Examination Solved Questions (Unit 1)

University Solved Examination Problems (MMDU Pattern)

Question 1 (1 Mark): Missing Intermediate Stage

Question: In the standard C compilation pipeline, complete the missing block [?] generated by compiler `cc1` prior to machine code generation:

```

prog.c → Preprocessor (cpp) → prog.i → Compiler (cc1) → [ ? ] → Assembler (as) → prog.o

```

Step-by-Step Solution:

```

prog.i (Expanded C) → Compiler (cc1) → Assembly Code (prog.s) → Assembler (as) → prog.o

```

The compiler `cc1` translates expanded pure C code into target ISA **Assembly Language Code** (`prog.s`).

Question 2 (1 Mark): Memory Segment Identification

Question: Refer to the virtual RAM architecture below. Which labeled segment corresponds to memory allocated via `malloc()` ?

```

[ High Memory: Stack (↓) ] → [ Free Memory Space ] → [ Segment X (↑) ] → [ BSS ] → [ Data ] → [ Text ]

```

Step-by-Step Solution:

Segment X = Heap Segment (Grows Upwards towards higher memory addresses)

Segment X is the Heap Segment, managed dynamically at runtime using functions in `stdlib.h`.

Question 13 (2 Marks): Dry Run Loop Trace

Question: Trace the variable states during execution of the digit-sum loop for `N = 345` :

```
while(n > 0) { rem = n % 10; sum += rem; n /= 10; }
```

Step-by-Step Solution:

Iteration	Initial <code>n</code>	Condition <code>n > 0</code>	<code>rem = n % 10</code>	<code>sum += rem</code>	Updated <code>n /= 10</code>
Init	345	-	-	0	345
Pass 1	345	True	5	$0 + 5 = 5$	34
Pass 2	34	True	4	$5 + 4 = 9$	3
Pass 3	3	True	3	$9 + 3 = 12$	0
Exit	0	False (Terminates)	-	12 (Final Result)	0

Question 17 (4 Marks): Quadratic Equation Algorithm & Flowchart

Question: Construct an algorithm and top-down decision structure to compute all roots of $ax^2 + bx + c = 0$.

Step-by-Step Solution:

- Ingest coefficients a, b, c .
- Compute Discriminant $D = b^2 - 4ac$.
- Decision Branches:
 - If $D > 0$: Real & Distinct roots: $r_1 = \frac{-b + \sqrt{D}}{2a}, r_2 = \frac{-b - \sqrt{D}}{2a}$.
 - If $D = 0$: Real & Equal roots: $r_1 = r_2 = \frac{-b}{2a}$.
 - If $D < 0$: Complex Conjugate roots: $extReal = \frac{-b}{2a}, Imag = \frac{\sqrt{-D}}{2a}$. Output: $extReal \pm i \cdot Imag$.

ANSI FLOWCHART: QUADRATIC EQUATION ROOTS RESOLUTION ($ax^2 + bx + c = 0$)

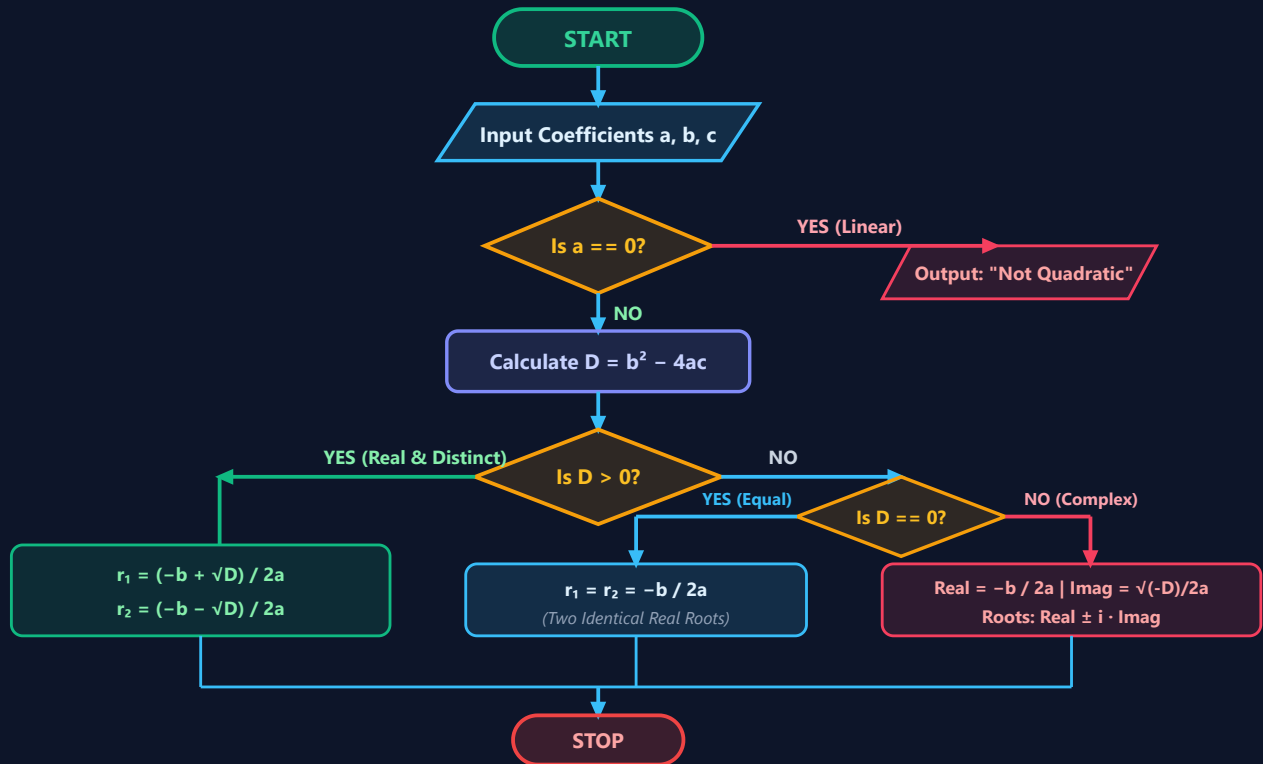


Fig 1.1: Complete ANSI Standard Decision Flowchart for Quadratic Equation Roots (Discriminant Analysis)

```

#include <stdio.h>
#include <math.h>

void solveQuadratic(double a, double b, double c) {
    if (a == 0) { printf("Not quadratic!\n"); return; }
    double d = b*b - 4*a*c;
    if (d > 0) {
        printf("Roots: %.2f, %.2f\n", (-b + sqrt(d))/(2*a), (-b - sqrt(d))/(2*a));
    } else if (d == 0) {
        printf("Equal Roots: %.2f\n", -b/(2*a));
    } else {
        printf("Complex: %.2f + %.2fi, %.2f - %.2fi\n", -b/(2*a), sqrttrac{-d}{2*a}, -b/(2*a), sqrttrac{-d}{2*a});
    }
}
  
```

Core Programming Concepts, Operators & Control Structures

Exam Weightage: 24 - 28 Marks • **Estimated Study Time:** 38 mins

C Tokens, 32 ANSI keywords, identifier grammar, constants, primitive data types table, 9-tier operator precedence hierarchy, implicit integer promotions vs explicit typecasting, header files taxonomy, switch fall-through, and loop lifecycles with nested 2D matrix traversal.

1. Tokens, Keywords, Identifiers & Primitive Data Types

1. C Tokens Architecture

Tokens are the atomic lexical units recognized by the C compiler. They comprise:

- Keywords:** 32 reserved words in C89/C90 (e.g., `auto` , `volatile` , `sizeof` , `typedef`).
- Identifiers:** User-defined symbolic names for variables and functions. Rules: Must begin with `[a-zA-Z_]` , followed by alphanumeric or underscore characters. Case-sensitive.
- Constants:** Immutable literals (Integer literals `100` , Octal `012` , Hex `0x64` , Floating-point `3.14` , Character `'A'` , String `"Hello"`).

2. Primitive Data Types, Memory Storage & Specifiers

Data Type	Storage in RAM	Format Specifier	Range (32-bit Architecture)	Encoding Scheme
char	1 Byte (8 bits)	%c	-128 to 127	ASCII / 2's Complement
unsigned char	1 Byte	%c , %u	0 to 255	Pure binary magnitude
int	4 Bytes (32 bits)	%d , %i	-2,147,483,648 to 2,147,483,647	Signed 2's Complement
unsigned int	4 Bytes	%u	0 to 4,294,967,295	Pure binary magnitude
float	4 Bytes	%f	$\approx 3.4 \times 10^{-38}$ to 3.4×10^{38} (6 decimals)	IEEE 754 Single Precision
double	8 Bytes	%lf	$\approx 1.7 \times 10^{-308}$ to 1.7×10^{308} (15 decimals)	IEEE 754 Double Precision

| **void*** | 4 / 8 Bytes | %p | Valid Virtual Memory Addresses | Unsigned Hexadecimal |

2. Complete 9-Tier Operator Precedence & Associativity Hierarchy

Complete 9-Tier Operator Hierarchy Master Table

Tier	Category	Operators	Associativity	Key Exam Rules & Behaviors				
1 (Highest)	Postfix / Subscript	<code>() [] -></code> <code>. ++ --</code>	Left-to-Right	Function call <code>()</code> , array subscript <code>[]</code> , and member access <code>-></code> resolve first.				
2	Unary	<code>+ - ! ~</code> <code>++ --</code> <code>(type) &</code> <code>sizeof</code>	Right-to-Left	Prefix operators, address-of <code>&</code> , dereference <code>*</code> , and explicit cast evaluate right-to-left.				
3	Multiplicative	<code>/ %</code>	Left-to-Right	Modulus <code>%</code> requires integral operands only (compiler error on floats).				
4	Additive	<code>+ -</code>	Left-to-Right	Standard arithmetic addition and subtraction.				
5	Relational	<code>< <= ></code> <code>>=</code>	Left-to-Right	Yields Boolean 1 (True) or 0 (False).				
6	Equality	<code>== !=</code>	Left-to-Right	Lower precedence than relational: <code>a < b</code> <code>== c</code> evaluates as <code>(a < b) ==</code> <code>c</code> .				
7	Logical	<code>&&</code> (AND) then <code>&</code>		(OR)	Left-to-Right	Short-Circuit: In <code>A && B</code> , if A=0, B is skipped! In <code>A</code>		B, if A=1, B is skipped!
8	Ternary Conditional	<code>? :</code>	Right-to-Left	<code>condition ?</code> <code>expr1 : expr2</code> . Nested ternaries group right-to-left.				

| **9 (Lowest)** | Assignment & Comma | `= += -= = /= %=` | `,` | **R-to-L** (`=`) | L-to-R (`,`) | Comma operator evaluates left-to-right and returns rightmost expression. |

3. Type Conversion: Implicit Promotion vs Explicit Typcasting

1. Implicit Type Conversion (Usual Arithmetic Conversions)

The C compiler automatically promotes lower-ranked types to higher-ranked types during binary expression evaluation to preserve precision:

```
bool → char → short → int → unsigned int → long → unsigned long → float → double → long double
```

⚠ **The Classic Integer Division Hazard:**

```
> int a = 5, b = 2;
> float result = a / b; // result evaluates to 2.000000, NOT 2.5!
>
```

Root Cause: Both `a` and `b` are integers. The compiler performs integer division (`5 / 2 = 2`), discarding the remainder before promoting the integer `2` to float `2.000000`.

2. Explicit Typcasting

The programmer explicitly overrides default conversions using the cast operator `(target_type)expression`:

```
int a = 5, b = 2;
float result = (float)a / b; // Promotes 'a' to float first → 5.0 / 2 yields 2.500000!
```

4. Standard Header Files Taxonomy

Directory of Key ANSI C Standard Library Headers

Header File	Primary Purpose & Declarations	Canonical Library Functions
<code><stdio.h></code>	Standard Input/Output Streams & File Operations	<code>printf</code> , <code>scanf</code> , <code>fopen</code> , <code>fscanf</code> , <code>fprintf</code> , <code>fclose</code> , <code>fgets</code>
<code><stdlib.h></code>	Dynamic Memory, Process Control, Conversions	<code>malloc</code> , <code>calloc</code> , <code>realloc</code> , <code>free</code> , <code>exit</code> , <code>atoi</code> , <code>rand</code>
<code><string.h></code>	Null-terminated string manipulation & memory blocks	<code>strlen</code> , <code>strcpy</code> , <code>strncpy</code> , <code>strcat</code> , <code>strcmp</code> , <code>memcpy</code> , <code>memset</code>
<code><math.h></code>	Mathematical & trigonometric computational functions	<code>pow</code> , <code>sqrt</code> , <code>sin</code> , <code>cos</code> , <code>tan</code> , <code>floor</code> , <code>ceil</code> , <code>fabs</code>
<code><ctype.h></code>	Character classification and case transformation	<code>isalpha</code> , <code>isdigit</code> , <code>isalnum</code> , <code>isspace</code> , <code>toupper</code> , <code>tolower</code>
<code><stdbool.h></code>	Standard Boolean definitions (C99 onwards)	<code>bool</code> , <code>true</code> (1), <code>false</code> (0)

`<limits.h>` | Architecture platform limits for primitive integers | `INT_MAX`, `INT_MIN`, `CHAR_BIT`, `LONG_MAX` |

5. Control Structures: Decision-Making, Loops & Jump Statements

1. Decision-Making & Switch Fall-Through Mechanics

In C `switch(expression)`, the expression must evaluate to an integral or character constant. Omitting the `break;` statement causes **Fall-Through**, where control cascades sequentially into subsequent cases:

```
switch (grade) {
    case 'A': printf("Excellent "); // Omitting break causes fall-through!
    case 'B': printf("Good ");      // Executes if grade is 'A' OR 'B'!
                break;
    default:  printf("Average");
}
```

2. Looping Lifecycles & Jump Statements

Loop Construct	Control Type	Minimum Iterations	Best Suited For
<code>for</code> Loop	Entry-Controlled (Condition evaluated before body)	0 times	Deterministic iterations with known step bounds.
<code>while</code> Loop	Entry-Controlled (Condition evaluated before body)	0 times	Event-driven loops where termination depends on runtime state.
<code>do-while</code> Loop	Exit-Controlled (Condition evaluated after body)	1 time	Interactive menu loops requiring at least one display pass.

ISI ARCHITECTURAL FLOWCHART: WHILE LOOP (ENTRY-CONTROLLED) VS DO-WHILE (EXIT-CONTROLLED)

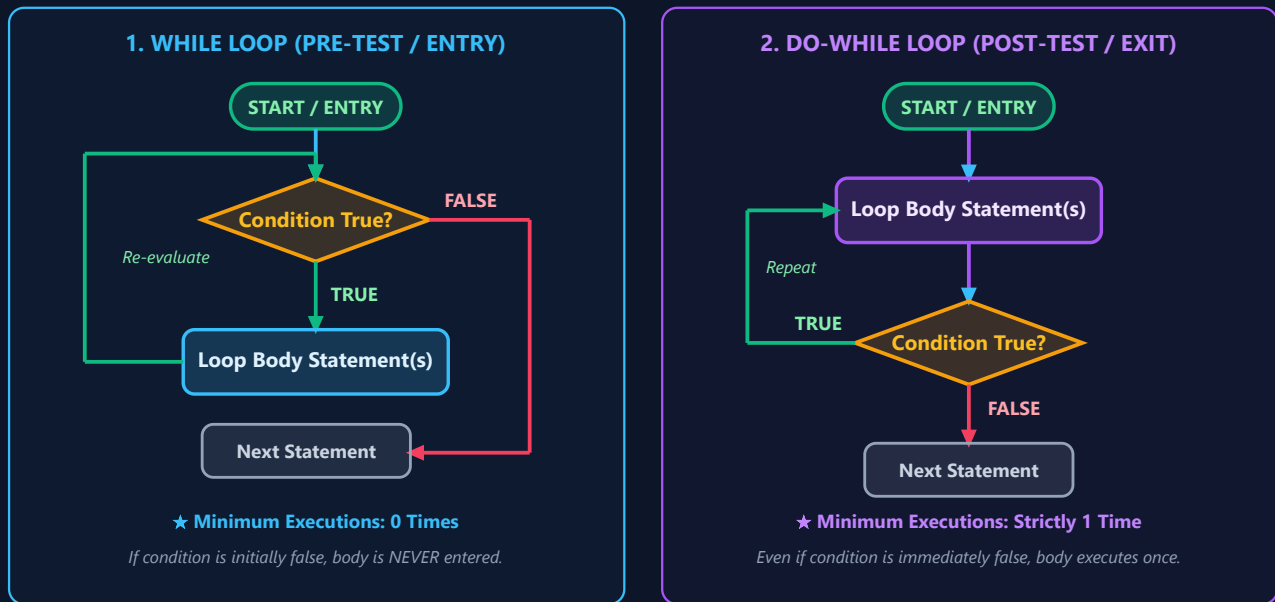


Fig 2.1: Architectural Control-Flow Comparison: Pre-Tested While Loop vs Post-Tested Do-While Loop

Jump Statements:

- `break;` : Instantly aborts the innermost enclosing loop or switch block.
- `continue;` : Skips the remaining statements of the current pass and jumps directly to the loop update expression (`i++`).
- `goto label;` : Unconditional jump directly to a labeled statement. Strongly discouraged in structured programming.

6. University Examination Solved Questions (Unit 2)

University Solved Examination Problems (MMDU Pattern)

Question 4 (1 Mark): Operator Precedence Evaluation

Question: Evaluate the expression order for: `int x = 2, y = 3; int z = x + y ++x;`

*Binary Operator Tree with root '+' having Left operand 'x' and right subtree 'y * (++x)'.*

Step-by-Step Solution:

1. Prefix increment `++x` (tier 2 unary) executes first: `x` becomes `3` .

2. Multiplicative operator `*` (tier 3) evaluates: `y x = 3 3 = 9` .

3. Additive operator `+` (tier 4) evaluates: `x + 9 = 3 + 9 = 12` .

Result: `z = 12` .

Question 14 (2 Marks): Short-Circuit Logic Evaluation

Question: What are the final values of `a` and `b` after executing:

```
int a = 0, b = 5;
if (a && ++b) { /* empty */ }
```

Step-by-Step Solution:

Left Operand (`a == 0`) is FALSE → Short-Circuit Triggered → Right expression (`++b`) is BYPASSED!

Under ANSI C rules for logical `&&` , if the left operand evaluates to `0` (false), the outcome is guaranteed false. The compiler bypasses evaluation of the right operand. Thus, `++b` is never executed, and `b` remains `5`.

Final Values: `a = 0, b = 5` .

Question 18 (4 Marks): Nested Loop Matrix Multiplication

Question: Explain the 3-tier nested loop structure required for multiplying matrices $A_{M \times K}$ and $B_{K \times N}$.

Step-by-Step Solution:

```
Outer Loop (i: 0 to M-1) → Iterates across Rows of Matrix A
Middle Loop (j: 0 to N-1) → Iterates across Columns of Matrix B
Inner Accumulator (k: 0 to K-1) → Dot product: C[i][j] += A[i][k] * B[k][j]
```

```
void multiplyMatrices(int A[2][2], int B[2][2], int C[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Complexity: Executes $M \times N \times K$ times, yielding time complexity $O(M \cdot N \cdot K)$ ($O(N^3)$ for square matrices).

Arrays, Strings, Functions & Modular Programming

Exam Weightage: 24 - 28 Marks • **Estimated Study Time:** 45 mins

1D & 2D arrays, mathematical derivation of row-major order addressing, Linear Search, Binary Search, Bubble Sort step trace, string library functions, 12-point storage classes matrix, and call by value vs reference.

1. 1D & 2D Arrays: Memory Mapping & Row-Major Formula Derivation

1. Array Memory Architecture

An array is a homogeneous collection of elements stored sequentially in contiguous memory locations.

- **1D Array Physical Address Formula:**

$$\text{extAddress}(A[i]) = \text{Base Address} + (i \times \text{sizeof}(T))$$

2. Mathematical Derivation: 2D Array Row-Major Order

In C, multi-dimensional arrays are mapped into linear RAM in **Row-Major Order** (row 0 stored first, followed by row 1, etc.).

For an array declared as `T A[R][C]` with base address `Base` and element size `S` :

- To locate element `A[i][j]` , the program must skip `i` full preceding rows.
- Each row contains exactly `C` columns (elements).
- Within row `i` , the program must skip `j` preceding column elements.

$$\text{extTotalElementsSkipped} = (i \times C) + j$$

$$\text{Address}(A[i][j]) = \text{Base Address} + \left[(i \times C) + j \right] \times \text{sizeof}(T)$$

✓ FUNDAMENTAL LAW / RULE:

Numerical Examination Example:

Given `int A[4][5]` starting at `Base = 2000` with `sizeof(int) = 4` bytes:

Find the physical address of element $A[2][3]$:

- $i = 2, C = 5, j = 3, S = 4$

- $extOffset = (2 \times 5 + 3) \times 4 = 13 \times 4 = 52 \text{ bytes}$

- $extAddress(A[2][3]) = 2000 + 52 = \text{mathbf{2052}}$

2. Searching & Sorting Algorithms: Linear, Binary & Bubble Sort

Searching & Sorting Master Comparative Matrix

Algorithm	Preconditions	Best Time	Worst Time	Space	Core Operational Principle
Linear Search	None (Unordered)	$O(1)$	$O(N)$	$O(1)$	Sequentially compares search key against indices 0 to $N - 1$.
Binary Search	Array must be sorted!	$O(1)$	$O(\log N)$	$O(1)$	Computes $mid = low + (high-low)/2$; halves search space each step.
Bubble Sort	None	$O(N)$ (optimized)	$O(N^2)$	$O(1)$	Repeatedly compares adjacent elements and swaps them, sinking maximum to the end.

Bubble Sort Step-by-Step Pass Trace

Tracing $[64, 34, 25, 12, 22, 11, 90]$:

- Initial: $[64, 34, 25, 12, 22, 11, 90]$
- Pass 1: Sinks **90** to index 6 $\rightarrow [34, 25, 12, 22, 11, 64, 90]$
- Pass 2: Sinks **64** to index 5 $\rightarrow [25, 12, 22, 11, 34, 64, 90]$
- Pass 3: Sinks **34** to index 4 $\rightarrow [12, 22, 11, 25, 34, 64, 90]$

- **Pass 4:** Sinks **25** to index 3 → [12, 11, 22, 25, 34, 64, 90]
- **Pass 5:** Sinks **22** to index 2 → [11, 12, 22, 25, 34, 64, 90]
- **Pass 6:** Zero swaps detected → **Array completely sorted in $O(N)$ optimized time!**

3. Exhaustive 12-Point Master Storage Classes Matrix

Complete 12-Point Storage Classes Matrix

Storage Class	Keyword	Storage Location	Initial Default	Scope / Visibility	Lifetime	Linkage	Exam Key Behavior
Automatic	auto	Stack Memory	Garbage value	Local to block	Block execution	None	Default for all local variables. Destroyed upon function return.
Register	register	CPU Microprocessor Register	Garbage value	Local to block	Block execution	None	Fast access hint for loop counters. Address-of &var is ILLEGAL!
Static	static	Data Segment (BSS / Data)	Zero (0 / '0')	Local to block or file	Entire program run	Internal	Retains value across function invocations. Initialized once at compile time.

| **External** | extern | Data Segment | Zero (0 / '0') | Global across all files | Entire program run | External | References a global variable defined in another file. Allocates no new memory. |

4. Parameter Passing: Call by Value vs Reference & Recursion

1. Call by Value vs. Call by Reference

Feature	Call by Value	Call by Reference
Passed Entity	Copy of actual argument value is passed.	Memory address of variable (&x) is passed via pointers.
Caller Modification	Mutations to formal parameters do NOT affect caller.	Dereferencing (p) <i>directly mutates caller variable.</i>
Memory Cost	Duplicates memory for large structs.	Zero copying overhead; passes a single 4/8-byte pointer.
C Syntax	void swap(int a, int b)	void swap(int a, int *b)

2. Recursion & Call Stack Frame Execution

Every recursive invocation allocates a new **Stack Frame** on the process stack. Execution must have a well-defined **Base Condition**; omitting it causes **Stack Overflow**:

```
Calling fact(3):  
[ Stack Frame 3: fact(3) waiting on 3 * fact(2) ] (Pushed)  
[ Stack Frame 2: fact(2) waiting on 2 * fact(1) ] (Pushed)  
[ Stack Frame 1: fact(1) hits base condition n <= 1 ] (Returns 1)  
----- Unwinding Phase -----  
Frame 2 receives 1 → computes 2 * 1 = 2 (Popped)  
Frame 3 receives 2 → computes 3 * 2 = 6 (Popped → Final Result = 6)
```

5. University Examination Solved Questions (Unit 3)

University Solved Examination Problems (MMDU Pattern)

Question 7 (1 Mark): 2D Row-Major Address Calculation

Question: In a row-major 2D array `int A[3][4]` starting at `Base = 1000` with `sizeof(int) = 4`, find the linear address of `A[1][2]`.

Step-by-Step Solution:

$$extOffset = (i \times C + j) \times \text{Size} = (1 \times 4 + 2) \times 4 = 6 \times 4 = 24 \text{ bytes}$$

$$extPhysicalAddress = 1000 + 24 = \text{mathbf{1024}}$$

Question 15 (2 Marks): Swap Implementation via Call by Reference

Question: Implement the classic integer swap function using pointers and explain why Call by Value fails.

Step-by-Step Solution:

```
// Call by Reference Swap:
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
} // In main: swap(&x, &y);
```

Call by Value passes copies of `x` and `y` ; mutating the copies leaves the caller's variables unchanged. Call by Reference passes memory addresses `&x` and `&y` , mutating actual values in the caller's stack frame.

Question 19 (4 Marks): Binary Search Implementation & Complexity

Question: Present the Binary Search algorithm and prove its $O(\log N)$ time complexity.

Step-by-Step Solution:

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents arithmetic overflow
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Not found
}
```

Proof: Each comparison step halves the remaining array interval: $N \rightarrow N/2 \rightarrow N/4 \dots \rightarrow N/2^k = 1$. Solving $2^k = N$ gives $k = \log_2 N$. Maximum comparisons are $O(\log N)$.

Advanced Data Handling, Pointers, Structures, DMA & File I/O

Exam Weightage: 24 - 28 Marks • Estimated Study Time: 45 mins

Address arithmetic, pointer taxonomy, Near, Far, and Huge Pointers (16-bit offset vs 32-bit segment:offset and auto-normalization), Struct vs Union memory alignment, Dynamic Memory Allocation (malloc, calloc, realloc, free), and File I/O streams.

1. Pointer Taxonomy: Wild, Dangling, Void & Function Pointers

1. Advanced Pointer Classification Matrix

Pointer Type	Syntax & Definition	Memory Hazard / Behavior	Remediation / Safe Practice
Wild Pointer	<code>int ptr;</code> (Uninitialized)	Points to arbitrary memory; dereferencing corrupts RAM or crashes process.	Always initialize to <code>NULL</code> : <code>int ptr = NULL;</code>
Dangling Pointer	<code>free(ptr);</code> (Target freed)	Points to deallocated storage that may be reassigned to other subsystems.	Set pointer to <code>NULL</code> immediately after freeing: <code>free(ptr); ptr = NULL;</code>
Void Pointer	<code>void vptr = &x;</code>	Generic memory pointer without data type association. Cannot be directly dereferenced.	Explicitly typecast before dereferencing: <code>(int)vptr</code>
Function Pointer	<code>int (fp)(int, int) = &add;</code>	Holds entry address of executable code in text segment. Enables runtime callbacks.	Invoked via <code>int res = fp(10, 20);</code>
Pointer to Array	<code>int (p)[5];</code>	Single pointer referencing an entire array block of 5 integers.	Used in multi-dimensional array parameter passing.

| **Array of Pointers** | `int p[5];` | Array containing 5 independent integer pointer variables. | Used in ragged 2D tables and string arrays (`char *argv[]`). |

2. Real-Mode 16-Bit Architecture: Near, Far, and Huge Pointers

8086 Segmented Memory Pointer Architecture

In 16-bit real-mode architecture (such as the 8086 microprocessor with 20-bit address bus addressing 1 MB of RAM), pointers are classified into three types:

$$extPhysicalAddress = (Segment\ Register \times 16) + Offset$$

Pointer Classification	Address Width	Internal Structure	Accessible Memory Span	Key Characteristics & Comparison Rules
Near Pointer	16-bit (2 Bytes)	Offset only	64 KB (within default DS/CS)	Fastest address arithmetic; cannot access data outside the current 64 KB segment.
Far Pointer	32-bit (4 Bytes)	16-bit Segment : 16-bit Offset	1 MB (Full real-mode RAM)	Can address any byte in 1 MB. Hazard: Addresses are unnormalized. <code>0x1000:0x0010</code> and <code>0x1001:0x0000</code> point to the identical byte, but <code>p1 == p2</code> evaluates to FALSE!

| **Huge Pointer** | **32-bit (4 Bytes)** | Normalized Segment : Offset | **1 MB** (Full real-mode RAM) | **Auto-Normalizing:** Any offset ≥ 16 is converted: $extSegment \leftarrow Segment + (Offset/16)$ and $extOffset \leftarrow Offset \bmod 16$. Relational comparisons are mathematically accurate across segments. |

3. Structures, Unions & Memory Alignment Architecture

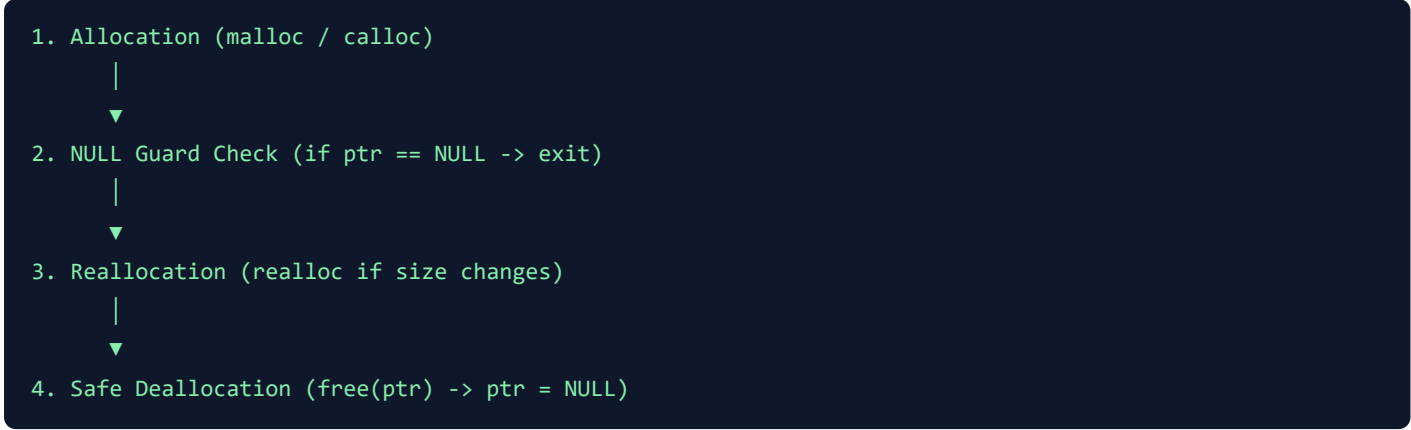
Structure vs. Union Architecture

Dimension	Structure (<code>struct</code>)	Union (<code>union</code>)
Keyword	<code>struct Record { int id; char grade; double score; };</code>	<code>union Record { int id; char grade; double score; };</code>
Memory Allocation	Sum of all member sizes + structure padding bytes.	Space equal only to the size of its largest member.
Memory Sharing	Each member has its own distinct, sequential address offset.	All members share the exact same starting memory address.
Active Members	All members can store valid values simultaneously.	Only ONE member can hold a valid value at any given instant.
Size Calculation	$extsizeof(struct) = 4 + 1 \text{ (pad 3)} + 8 = \textit{mathbf{16}}$ Bytes	$extsizeof(union) = \max(4, 1, 8) = \textit{mathbf{8}}$ Bytes

| **Member Access** | `var.id` (Direct) or `ptr->id` (Indirect via pointer) | `u.id` (Direct) or `u_ptr->id` (Indirect via pointer) |

4. Dynamic Memory Allocation (DMA) Complete Lifecycle

The Complete DMA Lifecycle



DMA Functions ():

- `malloc(size_t size)` : Allocates raw, uninitialized bytes from the heap pool (contains garbage).
- `calloc(size_t n, size_t size)` : Allocates memory and initializes all bits to zero.
- `realloc(void ptr, size_t new_size)` : Resizes an existing heap block without losing existing data.
- `free(void ptr)` : Returns memory back to the heap pool to prevent memory leaks.

5. File Handling Operations & Stream Modes

File Stream Modes & I/O Functions

Mode	Stream Meaning	If File Missing	If File Already Exists	Stream Functions
"r"	Read Text	Returns <code>NULL</code> error pointer	Opens file; read position at byte 0	<code>fscanf()</code> , <code>fgetc()</code> , <code>fgets()</code>
"w"	Write Text	Creates a new empty file	Truncates (erases) file to 0 bytes!	<code>fprintf()</code> , <code>fputc()</code> , <code>fputs()</code>
"a"	Append Text	Creates a new empty file	Preserves data; all writes appended at EOF	<code>fprintf()</code> , <code>fputs()</code>
"r+"	Read & Write	Returns <code>NULL</code> error pointer	Opens file for update without truncation	<code>fseek()</code> , <code>ftell()</code> , <code>rewind()</code>

| `"rb"` / `"wb"` | Binary Stream | Opens file for binary mode | No CR/LF translation; raw byte storage | `fread()` , `fwrite()` |

6. University Examination Solved Questions (Unit 4)

University Solved Examination Problems (MMDU Pattern)

Question 10 (1 Mark): Dangling Pointer Remediation

Question: What hazard is introduced after calling `free(p);` , and how is it resolved?

Step-by-Step Solution:

```
Pointer p → [ Address 0x5000 ] → Heap block released → Pointer p still retains 0x5000
```

`p` becomes a **Dangling Pointer**. Dereferencing it leads to memory corruption or crashes. Remediation: explicitly assign `p = NULL;` immediately after calling `free(p);` .

Question 20 (4 Marks): Near, Far, and Huge Pointers in 8086

Question: Explain Near, Far, and Huge pointers in 16-bit 8086 architecture with segment arithmetic.

Step-by-Step Solution:

- 1. Near Pointer:** 16-bit offset. Limited to current 64 KB segment. Fast arithmetic.
 - 2. Far Pointer:** 32-bit (16-bit segment : 16-bit offset). Addresses full 1 MB. Unnormalized: `0x1000:0x0010` and `0x1001:0x0000` point to the same physical byte, but relational equality evaluates to false!
 - 3. Huge Pointer:** 32-bit auto-normalized pointer. Converts any offset ≥ 16 : $extSegment \leftarrow Segment + (Offset/16)$ and $extOffset \leftarrow Offset \bmod 16$. Relational comparisons are mathematically accurate.
-

Question 23 (6 Marks): Student Records with DMA and Binary File I/O

Question: Design an architecture in ANSI C to dynamically allocate an array of `Student` structures on the heap, write records to `students.dat` , and read them back.

Step-by-Step Solution:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int rollNo;
    char name[32];
    float gpa;
} Student;

void processRecords(int n) {
    // 1. Dynamic Allocation
    Student *list = (Student*)malloc(n * sizeof(Student));
    if (!list) { printf("Heap allocation failed!");
    return; }

    // 2. Initialize sample record
    list[0].rollNo = 101; sprintf(list[0].name, "Aman"); list[0].gpa = 8.85f;

    // 3. Persist to Binary File
    FILE *fp = fopen("students.dat", "wb");
    if (fp) {
        fwrite(list, sizeof(Student), n, fp);
        fclose(fp);
    }

    // 4. Read back into fresh heap buffer
    Student *readList = (Student*)malloc(n * sizeof(Student));
    fp = fopen("students.dat", "rb");
    if (fp) {
        fread(readList, sizeof(Student), n, fp);
        printf("Recovered: Roll %d, Name: %s, GPA: %.2f",
            readList[0].rollNo, readList[0].name, readList[0].gpa);
        fclose(fp);
    }

    // 5. Clean up heap memory
    free(list); free(readList);
}

```

Question 24 (6 Marks): Function Pointers & Callback Sorting

Question: Explain the mechanism of Function Pointers in C by implementing a generic bubble sort that accepts a comparison callback.

Step-by-Step Solution:

```
#include <stdio.h>

int ascending(int a, int b) { return a > b; }
int descending(int a, int b) { return a < b; }

void genericSort(int arr[], int n, int (*compare)(int, int)) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (compare(arr[j], arr[j + 1])) { // Dynamic callback!
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int data[] = {45, 12, 89, 34};
    genericSort(data, 4, ascending); // Sorts [12, 34, 45, 89]
    genericSort(data, 4, descending); // Sorts [89, 45, 34, 12]
    return 0;
}
```